Southern New Hampshire University

CS 340 README

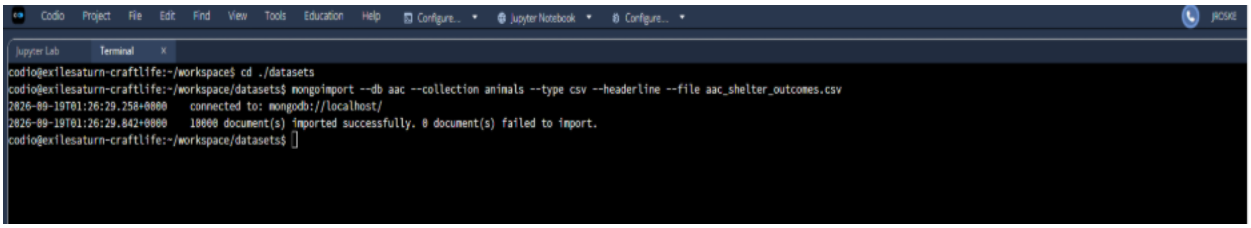
About the Project/Project Title
Grazioso Salvare Rescue Dog Identifier is a dashboard designed to help those who work in rescue dog training, identify dogs that meet the preferred characteristics of animals suitable for the job. The application is a full stack program that utilizes a python module and CRUD principles, to access a database of shelter animals and cleanly display the information on a dashboard interface.

Motivation
This application was designed for Grazioso Salvare, to assist them in their attempts at identifying suitable dogs for their rescue training program. For this, the organization has made agreements with local animal shelters, in the Austin, Texas region, to have access to their databases of animals. So, the organization wanted a program that they and other rescue training organizations may use to identify leading candidates for their training.

This application is intended to be that solution. It authorizes users access to these databases, allowing them the ability to create, read, update and delete records from the database, and do so from a clean and user-friendly dashboard, that is intended to increase the efficiency of searching for animals and reducing human error. From the dashboard, users can see a full data table of all the records, where they then can filter the results for the specific training needs, while also displaying a map of the region and a pie chart for the different dog breeds.

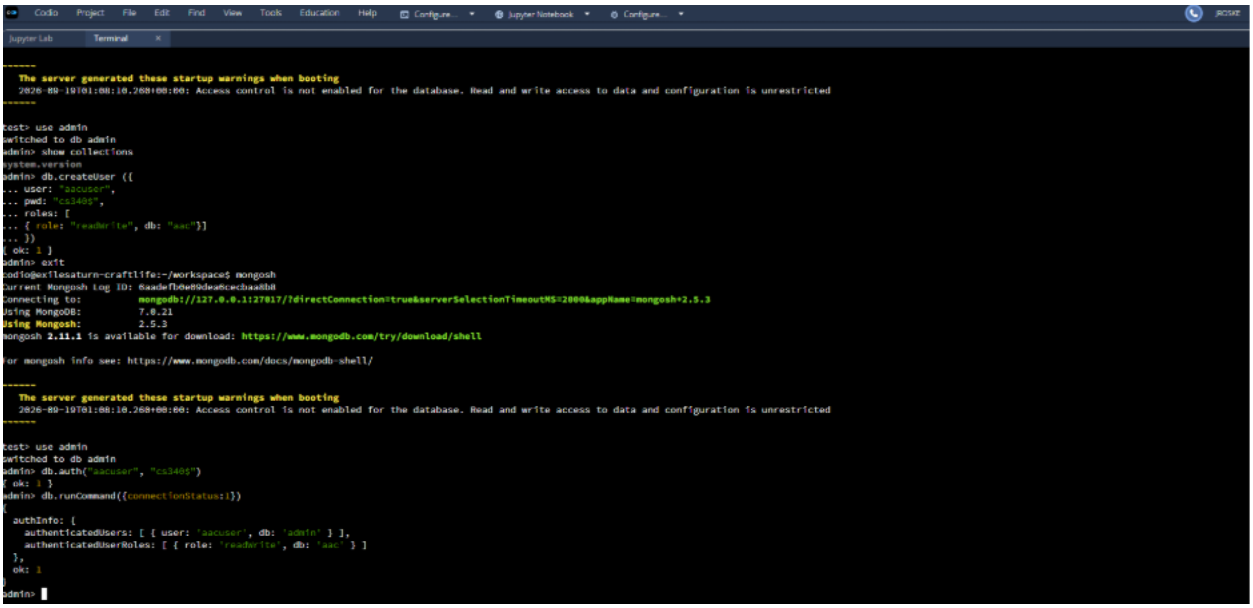
Getting Started
To get a local copy of this project up and running, the following criteria will need to be met. First, you need to have MongoDB downloaded on your machine, along with the current Python model, along with PyMongo and Dash libraries installed. From here, you will also need to have authorized access to the Austin Animal Center database and import it onto your local machine. Once these components are installed and running, you can download this applications source files and run the program.

Importing the AAC database
An important aspect of this application is the use of the AAC dataset. To gain access, the following steps are needed. In a terminal, you will need to run the following command.
Mongoimport --db aac --collection animals --type csv --headerline --file aacshelteroutcomes.csv



Once the database is imported, you will need to gain authorized access by using the following command.
Db.createUser ({
Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

user: “Username”,
pwd: “Password”,
roles: [
{role: “readwrite”, db: “aac”}]
})
Note enter your own username and password within the quotation marks.



Installation

- Python
  - PyMongo library
  - Plotly/Dash library
  - Pandas library
- MongoDB
- AAC database
- Copy this project into a python directory

Usage

Python Module

The Python module directly connects with the AAC database and utilizes the principles of CRUD, to interact and make changes to the database. For the overall program, this is the behind-the-scenes logic that bridges the gap between the dashboard and database, allowing the users to find the information they need.

Create Method

The following section shows the code base and testing used to develop the create method.

Code Example

The create method allows the user to create and add a new record into the database.

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

```python
# Complete this create method to implement the C in CRUD.
def create(self, data):
    try:
        if data is not None:
            self.database.animals.insert_one(data)  # data should be dictionary
            return True
        else:
            raise Exception("Nothing to save, because data parameter is empty")
    except Exception as e:
        return False
```

Tests

This test is designed to verify the creation of three test animals that are later used to test other methods. For this test specifically, it adds the three new animals to the database and only returns one to show the success of the method.

```python
#TODO: import for your CRUD module
from CRUD_Python_Module import AnimalShelter
#TODO: Instantiate an instance of your class
shelter = AnimalShelter('aacuser', 'cs340$')
#TODO: Use your create function create a new record in the aac database
test_animal_1 = {"rec_num" : 1, "age_upon_outcome" : "2 years", "animal_id" : "TEST_1", "animal_type": "Cat", "breed" : "Domestic Shot Hair",
                "color" : "Black", "date_of_birth" : "2014-04-10", "datetime" : "2017-4-11 09:00:00", "monthyear" : "2017-04-11T09:00:00",
                "name" : "", "outcome_subtype" : "SCRP", "outcome_type" : "Transfer", "sex_upon_outcome" : "Neutered Male", "location_lat" : 30.50,
                "location_long" : -97.424, "age_upon_outcome_in_weeks" : 156.77}
test_animal_2 = {"rec_num" : 2, "age_upon_outcome" : "2 years", "animal_id" : "TEST_2", "animal_type": "Cat_Test", "breed" : "Domestic Shot Hair",
                "color" : "Black", "date_of_birth" : "2014-04-10", "datetime" : "2017-4-11 09:00:00", "monthyear" : "2017-04-11T09:00:00",
                "name" : "", "outcome_subtype" : "SCRP", "outcome_type" : "Transfer", "sex_upon_outcome" : "Neutered Male", "location_lat" : 30.50,
                "location_long" : -97.424, "age_upon_outcome_in_weeks" : 156.77}
test_animal_3 ={"rec_num" : 3, "age_upon_outcome" : "2 years", "animal_id" : "TEST_3", "animal_type": "Cat_Test", "breed" : "Domestic Shot Hair",
                "color" : "Black", "date_of_birth" : "2014-04-10", "datetime" : "2017-4-11 09:00:00", "monthyear" : "2017-04-11T09:00:00",
                "name" : "", "outcome_subtype" : "SCRP", "outcome_type" : "Transfer", "sex_upon_outcome" : "Neutered Male", "location_lat" : 30.50,
                "location_long" : -97.424, "age_upon_outcome_in_weeks" : 156.77}

result = shelter.create(test_animal_1)
result = shelter.create(test_animal_2)
result = shelter.create(test_animal_3)
```

Read Method

This section shows the implementation and testing of the modules read functionality.

Code Example

The read method is designed to search the database for existing records and return the results that match the search criteria.

```python
# Create method to implement the R in CRUD.
def read(self, data):
    try:
        result = list(self.database.animals.find(data))
        return result
    except Exception as e:
        return list()
```

Tests

This test calls the read method and searches for one of the test animals, using a unique identifier to confirm the effectiveness of the method.

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

```
found = shelter.read({"animal_id":"TEST_1"})
print(found)
```

Update Method

This section shows the codebase and testing for the update method

Code Example

The update method is used to update existing records within the database. Since users may want to update one or more records, the method includes a branch that allows the user to choose between updating a single record or many.

```
    # Create method to implement the U in CRUD.
def update(self, data, new_data, many=False):
    try:
        update_values = {"$set" : new_data}
        if many is False:
            result = self.database.animals.update_one(data, update_values)
            return result.modified_count
        else:
            result = self.database.animals.update_many(data, update_values)
            return result.modified_count
    except Exception as e:
        return 0
```

Tests

The test for this method verifies that both branches of the method work, so it is called twice to ensure that the updateOne() and updateMany() functions work as intended.

```
#TODO: Use the update function for both one and many
updated = shelter.update({"animal_id" : "TEST_1"}, {"animal_id" : "TEST_01"}, False)
print(updated)

updated_many = shelter.update({"animal_type" : "Cat_Test"}, {"animal_type" : "Dog_Test"}, True)
print(updated_many)
```

Delete Method

This sections shows the implementation of the delete method and the testing used to verify that it works.

Code Example

This method is used to search through the database for an existing record or records and remove them from the database, returning the number of records removed.

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

```
    # Create method to implement the D in CRUD
def delete(self, data, many=False):
    try:
        if many is False:
            result = self.database.animals.delete_one(data)
            return result.deleted_count
        else:
            result = self.database.animals.delete_many(data)
            return result.deleted_count
    except Exception as e:
        return 0
```

Tests

This test shows the delete method in action and verifies the method works as intended. It is called twice to show that both the deleteOne() and deleteMany() functions work properly.

```
#TODO: use the delete function
deleted = shelter.delete({"animal_id" : "TEST_1"}, False)
print(deleted)

deleted_many = shelter.delete({"animal_type" : "Dog_Test"}, True)
print(deleted_many)
```

Screenshots

Below is a screenshot of the results of all the previously mentioned methods and tests in action.



```
True
[{'_id': ObjectId('6ab82a480d6fecf4d361ee14'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Blac
k', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_ou
tcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ab82a640d6fecf4d361ee16'), 'rec_num': 1, 'age_upon_out
come': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthy
ear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'ag
e_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ab82a750d6fecf4d361ee18'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Dom
estic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome
_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ac011700 5c807ecbf4337a
c'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetim
e': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 3
0.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ac011f205c807ecbf4337b0'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1',
'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '',
'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_i
d': ObjectId('6ac0123905c807ecbf4337b4'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'da
te_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome':
'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ac0127105c807ecbf4337b8'), 'rec_num': 1, 'age_upon_outcome':
'2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear':
'2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon
_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ac012e805c807ecbf4337bc'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic
Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_typ
e': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ac0130c03efb9f55d801b06'),
'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2
017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'l
ocation_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ac01324a3832b2757c8e27d'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_
type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome
_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': Object
Id('6ac01418a3832b2757c8e281'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birt
h': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered
Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}, {'_id': ObjectId('6ac014dfa3832b2757c8e285'), 'rec_num': 1, 'age_upon_outcome': '2 years',
'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair', 'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T
09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfer', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_
weeks': 156.77}, {'_id': ObjectId('6ac014f4a3832b2757c8e289'), 'rec_num': 1, 'age_upon_outcome': '2 years', 'animal_id': 'TEST_1', 'animal_type': 'Cat', 'breed': 'Domestic Shot Hair',
'color': 'Black', 'date_of_birth': '2014-04-10', 'datetime': '2017-4-11 09:00:00', 'monthyear': '2017-04-11T09:00:00', 'name': '', 'outcome_subtype': 'SCRP', 'outcome_type': 'Transfe
r', 'sex_upon_outcome': 'Neutered Male', 'location_lat': 30.5, 'location_long': -97.424, 'age_upon_outcome_in_weeks': 156.77}]
1
2
1
2
```


Dashboard

The dashboard utilizes Python and the third-party library from Jupyter, to create an interactive user interface, that allows users to search for animals that can be specialized for training in specialized forms of search and rescue.

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

Code Example

The following code is used to implement the filter buttons and filter out the preferred traits for specific kinds of rescue training. Uses a branching system, that receives the input from the button clicked, and returns the results that meet the criteria of the query.

```
@app.callback(
    Output('datatable-id', "data"),
    [Input('submit-button-one', 'n_clicks'),
     Input('submit-button-two', 'n_clicks'),
     Input('submit-button-three', 'n_clicks'),
     Input('submit-button-four', 'n_clicks')
    ])

def on_click(button1, button2, button3, button4):
    if ctx.triggered_id == "submit-button-one":
        df = pd.DataFrame.from_records(shelter.read({"animal_type" : "Dog", "breed" :{'$in' :
            ["Labrador Retriever Mix", "Chesapeake Bay Retriever", "Newfoundland"]},
            "sex_upon_outcome" : "Intact Female", "age_upon_outcome_in_weeks" :{ '$gte' : 26,'$lte' : 156}}))
    elif ctx.triggered_id == "submit-button-two":
        df = pd.DataFrame.from_records(shelter.read({"animal_type" : "Dog", "breed" : {'$in' :
            ["German Shepherd", "Alaskan Malamute", "Old English Sheepdog", "Siberian Husky", "Rottweiler"]},
            "sex_upon_outcome" : "Intact Male", "age_upon_outcome_in_weeks" : {'$gte' : 26, '$lte' : 156}}))
    elif ctx.triggered_id == "submit-button-three":
        df = pd.DataFrame.from_records(shelter.read({"animal_type" : "Dog", "breed" : {'$in' :
            ["Doberman Pinscher", "German Shepherd", "Golden Retriever", "Bloodhound", "Rottweiler"]},
            "sex_upon_outcome" : "Intact Male", "age_upon_outcome_in_weeks" : {'$gte' : 20, '$lte' : 300}}))
    elif ctx.triggered_id == "submit-button-four" :
        df = pd.DataFrame.from_records(shelter.read({}))

    else:
        df = pd.DataFrame.from_records(shelter.read({}))

    df.drop(columns=['_id'], inplace=True)
    return df.to_dict('records')
```

To handle the pie chart, the following code uses the inforamtion from the database and updates the chart according to the filter results.

```
@app.callback(
    Output('graph-id', "children"),
    [Input('datatable-id', "derived_virtual_data")])
def update_graphs(viewData):
    ###FIX ME ####
    # add code for chart of your choice (e.g. pie chart) #
    if not viewData:
        return [html.P("No animals to display.")]
    df = pd.DataFrame.from_records(viewData or [])

    return [
        dcc.Graph(
            figure = px.pie(df, names='breed', title='Preferred Animals')
        )
    ]
```

Similar to the above, the following code handles the map display and updates the map according to the record that is selected.

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

```
@app.callback(
    Output('map-id', "children"),
    [Input('datatable-id', "derived_virtual_data"),
     Input('datatable-id', "derived_virtual_selected_rows")])
def update_map(viewData, index):
    if viewData is None:
        return
    elif index is None:
        return

    dff = pd.DataFrame.from_dict(viewData)
    # Because we only allow single row selection, the list can be converted to a row index here
    if index is None:
        row = 0
    else:
        row = index[0]

    # Austin TX is at [30.75,-97.48]
    return [
        dl.Map(style={'width': '1000px', 'height': '500px'}, center=[30.75,-97.48], zoom=10, children=[
            dl.TileLayer(id="base-layer-id"),
            # Marker with tool tip and popup
            # Column 13 and 14 define the grid-coordinates for the map
            # Column 4 defines the breed for the animal
            # Column 9 defines the name of the animal
            dl.Marker(position=[dff.iloc[row,13],dff.iloc[row,14]], children=[
                dl.Tooltip(dff.iloc[row,4]),
                dl.Popup([
                    html.H1("Animal Name"),
                    html.P(dff.iloc[row,9])
                ])
            ])
        ])
    ]
```

Tests

Below is a working example of the dashboard. It shows the data table and graph as the program starts and shows the results of when the different filters are selected.

Dashboard upon opening and resetting the filter

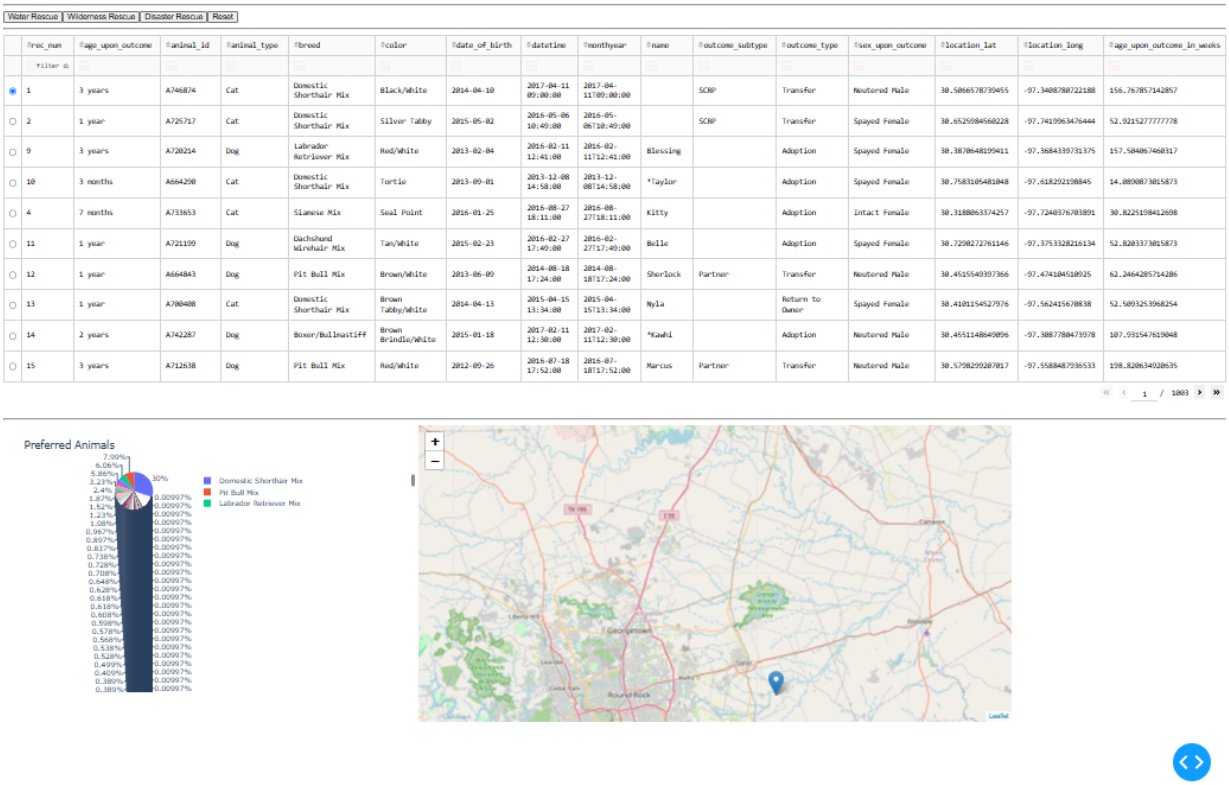
Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

Southern New Hampshire University

CS-340 Dashboard

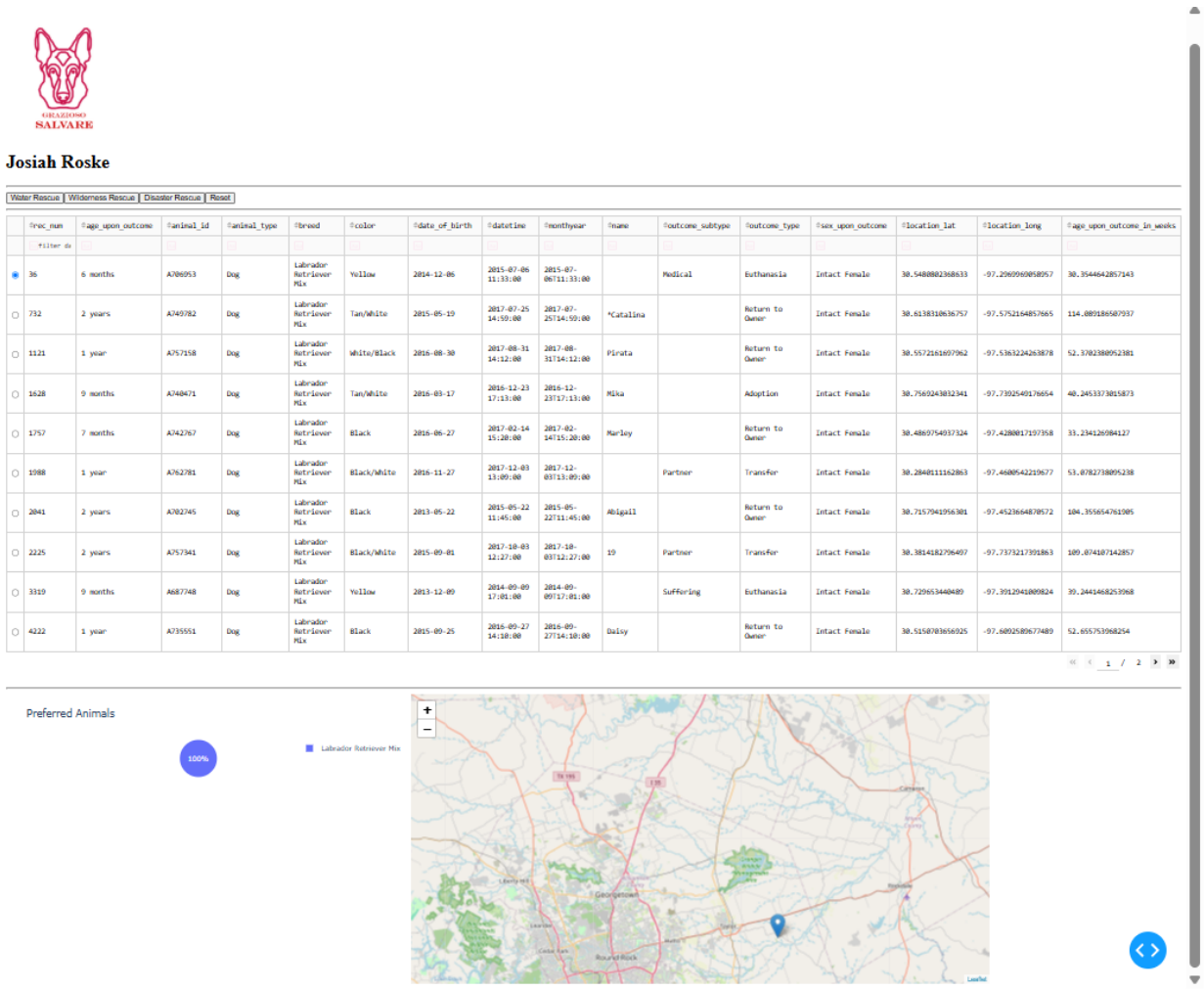


Josiah Roske



Dashboard after selecting Water Rescue

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.



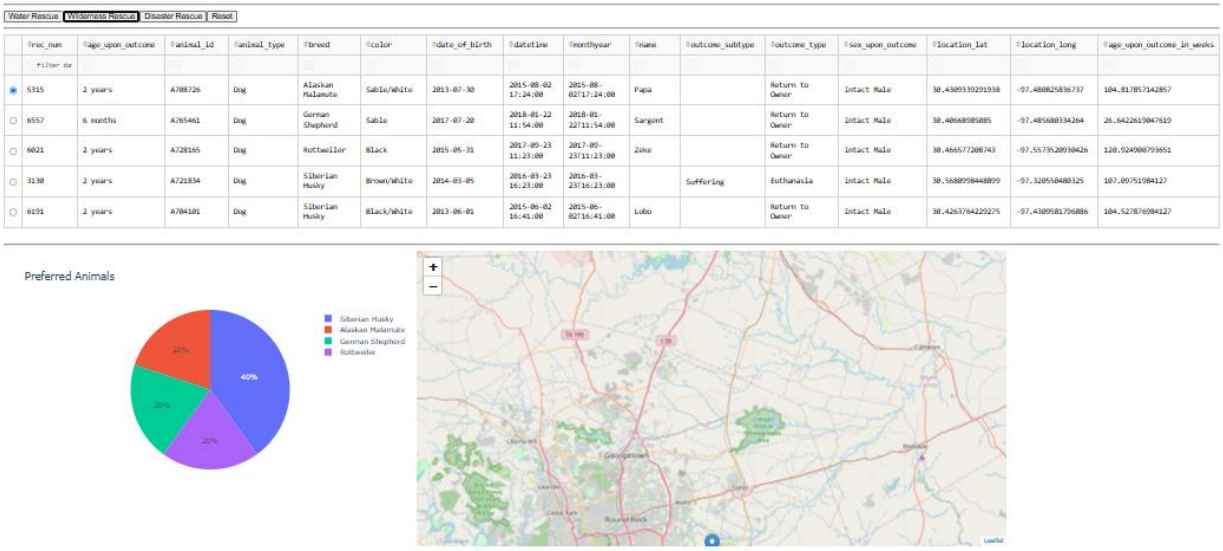
Dashboard after selecting Wilderness Rescue

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

Southern New Hampshire University

CS-340 Dashboard

Josiah Roske



Dashboard after selecting Disaster Rescue

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.

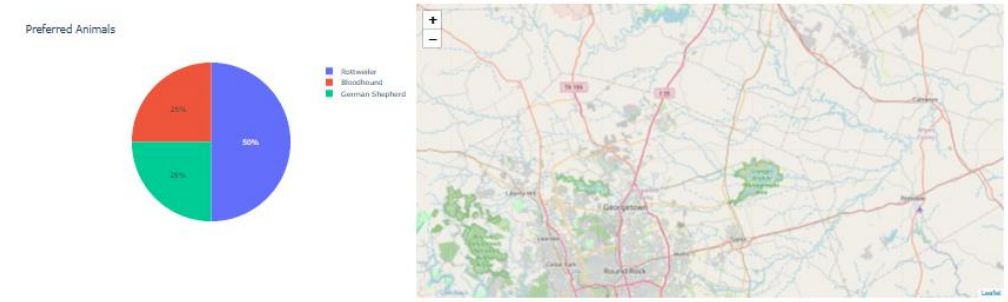
Southern New Hampshire University

CS-340 Dashboard

Josiah Roske

Water Rescue | Wilderness Rescue | Disaster Rescue | Reset

	rec_num	age_upon_outcome	animal_id	animal_type	breed	color	date_of_birth	datetime	monthyear	name	outcome_subtype	outcome_type	sex_upon_outcome	location_lat	location_long	age_upon_outcome_in_weeks
	filter da															
◉	3767	4 years	A712291	Dog	Bloodhound	Red	2011-09-20	2015-09-22 15:43:00	2015-09-22T15:43:00	Boomer		Return to Owner	Intact Male	30.2709983761287	-97.5923916912722	209.093551587302
○	6557	6 months	A765461	Dog	German Shepherd	Sable	2017-07-20	2018-01-22 11:54:00	2018-01-22T11:54:00	Sargent		Return to Owner	Intact Male	30.40668085085	-97.485688834264	26.6422619047619
○	2087	4 years	A694614	Dog	Rottweiler	Black/Brown	2011-01-01	2015-01-01 14:25:00	2015-01-01T14:25:00	Striker		Return to Owner	Intact Male	30.329873203611	-97.5492968638502	208.800099206349
○	6821	2 years	A728165	Dog	Rottweiler	Black	2015-05-31	2017-09-23 11:23:00	2017-09-23T11:23:00	Zeke		Return to Owner	Intact Male	30.4665772087743	-97.5573520930426	120.924008793651



Contact
Josiah Roske

Note: This template has been adapted from the following sample templates: Make a README, Best README Template, and A Beginners Guide to Writing a Kickass README.